

Souvenirs

Amaru cumpără suveniruri într-un magazin din străinătate. Există N **tipuri** de suveniruri, câte o infinitate din fiecare.

Fiecare suvenir are un preț fix. Mai exact, un suvenir de tipul i ($0 \leq i < N$) are un preț de $P[i]$ kosoni, unde $P[i]$ este un număr întreg pozitiv.

Amaru știe că suvenirurile sunt ordonate descrescător după preț și că prețurile sunt distincte. Anume, $P[0] > P[1] > \dots > P[N-1] > 0$. Mai mult, el știe valoarea $P[0]$. Din păcate, Amaru nu are alte informații legate de prețurile suvenirurilor.

Ca să cumpere suveniruri, Amaru va efectua un număr de tranzacții cu vânzătorul.

Fiecare tranzacție constă în următorii pași:

1. Amaru oferă un număr (pozitiv) de kosoni vânzătorului.
2. Vânzătorul pune kosonii într-o grămadă în camera din spate, unde Amaru nu o poate vedea.
3. Vânzătorul trece prin toate suvenirurile, în ordinea $0, 1, \dots, N-1$, în această ordine, unul câte unul. Fiecare suvenir este luat în considerare **exact o dată** per tranzacție.
 - Fiind la suvenirul de tip i , dacă numărul de kosoni din grămadă este cel puțin $P[i]$, atunci
 - vânzătorul extrage $P[i]$ kosoni din grămadă, și
 - vânzătorul pune un suvenir de tipul i pe masă.
4. Vânzătorul dă lui Amaru toți kosonii rămași în grămadă și toate suvenirurile de pe masă.

Observați că înainte de a începe o tranzacție, nu există niciun koson și niciun suvenir pe masă.

Ce trebuie să faceți este să-l instruiți pe Amaru să facă un număr de tranzacții, astfel încât:

- în orice tranzacție, el cumpără **cel puțin un** suvenir, și
- în total el cumpără **exacts** i suveniruri de tipul i , pentru fiecare i de la 0 la $N-1$. Observați că asta înseamnă că Amaru nu are voie să cumpere niciun suvenir de tipul 0.

Amaru nu trebuie să minimizeze numărul de tranzacții și are la dispoziție oricâți kosoni.

Detalii de implementare

Trebuie să implementați următoarea procedură.

```
void buy_souvenirs(int N, long long P0)
```

- N : numărul de suveniruri.
- $P0$: valoarea lui $P[0]$.
- Această procedură este apelată exact o dată în cadrul unui test.

Procedura de mai sus poate apela funcția de mai jos pentru a-l instrui pe Amaru cum să efectueze o tranzacție:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : Numărul de kosoni pe care Amaru îi dă vânzătorului.
- Funcția returnează o pereche. Primul element al perechii este un vector L , ce conține tipul suvenirurilor cumpărate (în ordine crescătoare). Al doilea element al perechii este un număr întreg R , reprezentând numărul de kosoni înapoiați lui Amaru în urma tranzacției.
- Este necesar ca $P[0] > M \geq P[N - 1]$. Condiția că $P[0] > M$ asigură că Amaru nu cumpără niciun suvenir de tipul 0, iar $M \geq P[N - 1]$ asigură că Amaru cumpără cel puțin un suvenir. Dacă aceste condiții nu sunt respectate, soluția ta va primi verdictul: `Output isn't correct: Invalid argument`. Observați că, spre deosebire de valoarea lui $P[0]$, valoarea lui $P[N - 1]$ nu este oferită în input.
- Funcția poate fi apelată de maximum 5000 de ori în cadrul unui test.

Comportamentul grader-ului **nu este adaptiv**. Asta înseamnă că secvența de prețuri P este fixată înainte de apelul `buy_souvenirs`.

Restricții

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ pentru fiecare i astfel încât $0 \leq i < N$.
- $P[i] > P[i + 1]$ pentru fiecare i astfel încât $0 \leq i < N - 1$.

Subtask-uri și punctare

Subtask	Scor	Restricții suplimentare
1	4	$N = 2$
2	3	$P[i] = N - i$ pentru fiecare i astfel încât $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ pentru fiecare i astfel încât $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ pentru fiecare i astfel încât $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ pentru fiecare i astfel încât $0 \leq i < N - 1$.
6	33	Fără restricții suplimentare.

Exemplu

Considerați următorul apel.

```
buy_souvenirs(3, 4)
```

Sunt $N = 3$ tipuri de suveniruri și $P[0] = 4$. Observați că există doar trei șiruri posibile de prețuri P : $[4, 3, 2]$, $[4, 3, 1]$, și $[4, 2, 1]$.

Să presupunem că `buy_souvenirs` apelează `transaction(2)`. Să presupunem că apelul returnează $([2], 1)$, Asta însemnând că Amaru cumpără un suvenir de tipul 2 iar vânzătorul îi dă înapoi 1 koson. Observați că asta ne ajută să deducem că $P = [4, 3, 1]$, deoarece:

- Pentru $P = [4, 3, 2]$, `transaction(2)` ar fi returnat $([2], 0)$.
- Pentru $P = [4, 2, 1]$, `transaction(2)` ar fi returnat $([1], 0)$.

Apoi `buy_souvenirs` poate apela `transaction(3)`, ce va returna $([1], 0)$, însemnând ca Amaru a cumpărat un suvenir de tipul 1 iar vânzătorul i-a dat înapoi 0 kosoni. Până acum, în total, el a cumpărat un suvenir de tipul 1 și un suvenir de tipul 2.

Într-un final, `buy_souvenirs` poate apela `transaction(1)`, ce va returna $([2], 0)$, însemnând că Amaru a cumpărat un suvenir de tipul 2. Observați că puteam apela aici și `transaction(2)`. În acest punct, în total Amaru are un suvenir de tipul 1 și două suveniruri de tipul 2, după cum era cerut.

Sample Grader

Formatul de intrare:

N

$P[0] \quad P[1] \quad \dots \quad P[N-1]$

Formatul de ieșire:

$Q[0] \quad Q[1] \quad \dots \quad Q[N-1]$

Aici $Q[i]$ reprezintă numărul de suveniruri de tipul i cumpărate în total pentru fiecare i astfel încât $0 \leq i < N$.